WHR

语言解释器与综合应用

从语言设计，到可运行、可测试、可复现、可讲解

程序设计实践大作业 · WHR v1.0 · 2026

解释器 | 64 项黑盒测试 | 267 行应用 | Agent Skill

作业要求，全部落到可检查证据

评分项	实现证据	现场检查
解释器	词法→语法→ AST→ 执行	python -m whr
自动测试	64 个进程级黑盒用例	python -m tests.run_all
性能比较	2 负载 × 2 实现 × 7 轮	JSON 原始结果 + 报告
人类与 AI 指南	4 份指南 + 项目 Skill	独立 Agent 前向验证
200+ 行应用	校园生存模拟器	267 有效行 + 4 条结局

关键原则：每一项交付物都有命令、数据或 Git 历史可以复核。

WHR 的语言设计：简单、明确、又有自己的辨识度

一段代码看懂核心语法

```
form Student { name, score }
craft grade(student) {
  when student.score >= 60 {
    give "pass"
  } otherwise { give "retry" }
}
let roster <- []
push(roster, Student { "Ada", 92 })
say grade(roster[0])
```

`<-`	赋值方向清晰
`form`	声明结构类型
`craft / give`	定义函数 / 返回值
`when / otherwise`	分支控制
`repeat while`	循环控制

语义选择：动态类型、词法作用域、严格布尔条件、短路逻辑。

解释器架构：一条可逐层讲清的执行流水线



每层只做一件事

Lexer 识别词法；Parser 按优先级生成 AST ；Interpreter 在 Environment 中执行节点。

优点：结构清晰、错误定位明确、特别适合课程验收时逐层演示。

运行时语义：从基本类型到闭包

基本值 整数 · 字符串 · 布尔 · none

容器 数组创建、索引、更新、 push/pop

结构 form 声明、字段读取与赋值

函数 参数、返回、递归、词法闭包

严格布尔： when 与 repeat while 只接受 true / false

错误不是崩溃：统一诊断、稳定退出码

LexError

非法字符、字符串未闭合

源位置

ParseError

语法结构不完整

期望 Token

RuntimeError

类型、索引、字段、调用错误

运行位置

```
$ python -m whr bad.whr

RuntimeError [line 4, column 9]: array index 5 out of range
```

退出码：0 成功 · 1 语言错误 · 2 命令行错误

测试策略：从 .whr 源程序到进程结果的黑盒验证

64 / 64
黑盒用例通过

31 / 31
Python 单元 / 集成测试

0
正式验证失败

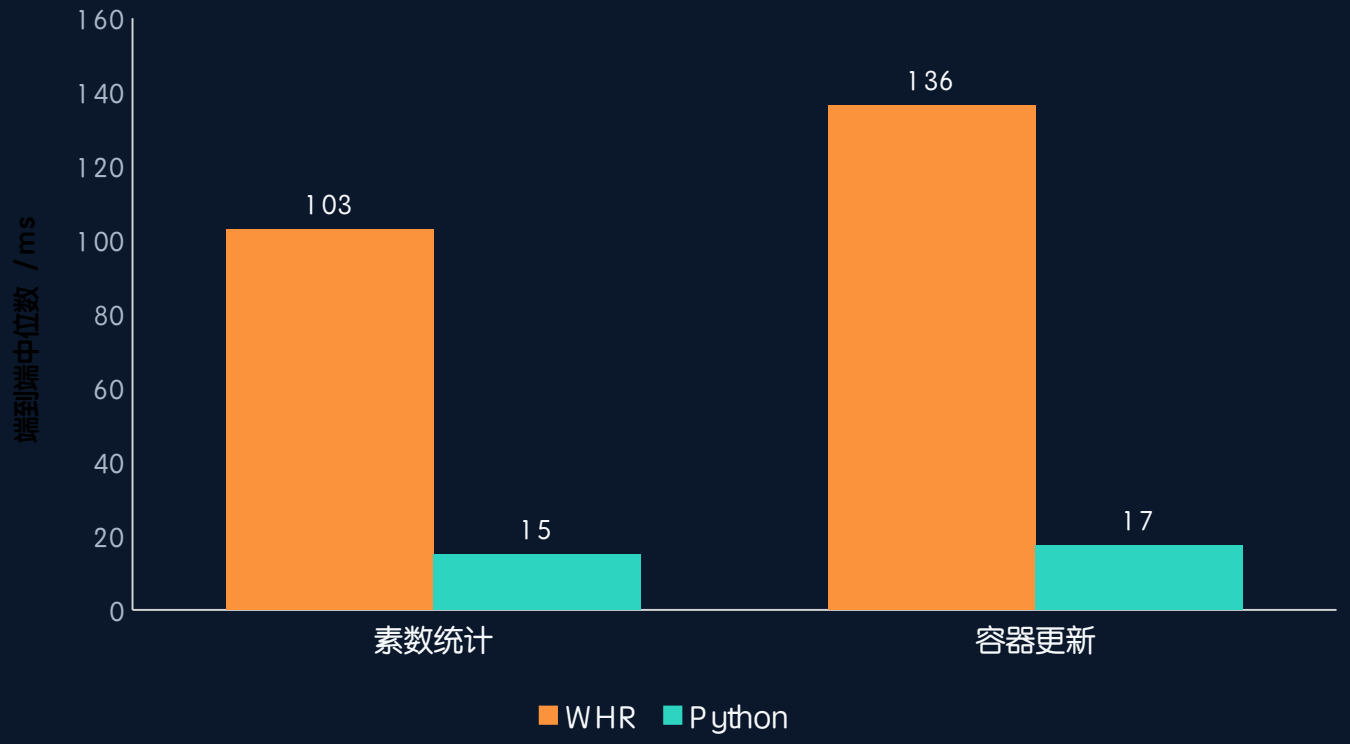
1 条命令
运行全部黑盒测试

类型	数组	结构	I/O	控制流	函数	错误	回归 +CLI
8	5	5	6	6	8	22	4

每个用例声明期望 stdout / stderr / exit code ；测试驱动器启动真实解释器进程。

这证明的是用户看到的行为，而不是内部函数“碰巧通过”。

性能测试：公平、留痕，也诚实面对差距



6.83×

素数统计倍率

7.83×

容器更新倍率

2 次预热 · 7 轮正式计时 · 交替执行顺序 · 0 失败

结论：树遍历解释器更慢；数据用于定位代价，不掩饰结果。

让 Agent 会写 WHR：指南不是摆设，而是可验证接口

01
读规范
语法与错误模型

02
按模板实现
任务、约束、验收

03
本地验证
--check 与真实运行

04
交付证据
源码、输出、修改记录

独立前向测试

新 Agent 只读取项目 Skill，就独立完成结构体、数组、函数与循环示例，并通过真实解释器。

证据：`.agents/skills/whr-language/` + `docs/agent_logs/06_skill_forward_test.md`

267 行综合应用：校园生存模拟器

玩家连续 6 周选择行动，在绩点、精力、压力、人脉与金币之间权衡。

GPA　ENERGY　STRESS　SOCIAL　COINS

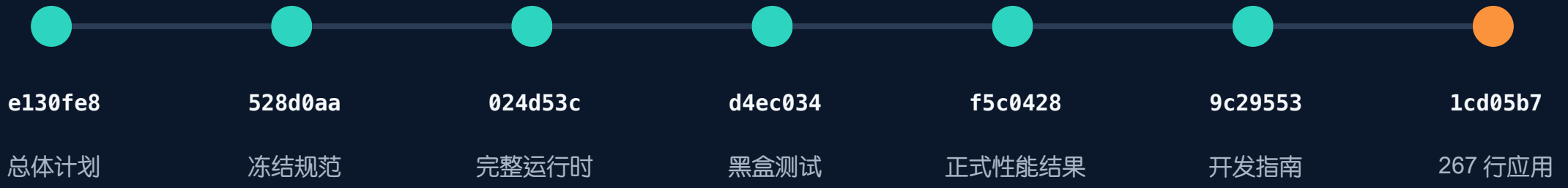
菜单输入 → 行动结算 → 随机事件 → 状态约束 → 结局判定

267
有效 WHR 代码行

4
可达结局均有测试

6
应用级自动测试

可复现交付：阶段文档、语义提交、统一验收




```
$ python scripts/check_project.py
```

一次检查：交付物 · CLI · 黑盒 · 单元测试 · LOC · 性能证据 · PPT

每阶段至少一个 Git 提交，规范、实现、测试和文档可以沿历史回看。

现场演示顺序：5 分钟证明项目闭环

1	运行 language_tour.whr	证明核心语法
2	运行 64 项黑盒测试	证明完整性
3	演示校园生存模拟器	证明综合使用
4	展示性能 JSON 与报告	证明实验可复现
5	执行统一项目检查	证明交付闭环

结论：WHR 不只是“能跑”，而是有规范、有证据、能复现、能扩展。